Recursive Function to Find the GCD of Two Positive Integers

using Euclid's Algorithm

Name	**:**	Rudra Pratap Singh
SAP ID	**:**	590032811
Batch	**:**	16

1. Problem Statement

Write a recursive function to find the Greatest Common Divisor (GCD) of two positive integers using Euclid's algorithm.

2. Objective

- To understand the concept of recursion and how a function can call itself.
- To apply Euclid's algorithm to compute the GCD of two numbers efficiently.
- To implement and test the solution as a C program.

3. Theory

GCD (Greatest Common Divisor) of two positive integers is the largest integer that divides both of them without leaving a remainder. For example, the GCD of 48 and 18 is 6.

Euclid's algorithm is based on the principle that the GCD of two numbers does not change if the larger number is replaced by its remainder when divided by the smaller number. Mathematically:

```
gcd(a, b) = a,                  if b = 0
gcd(a, b) = gcd(b, a mod b),    if b > 0
```

Recursion is a technique in which a function calls itself to solve a smaller instance of the same problem. A recursive function needs two parts:

- **Base case:** the condition where the recursion stops (here, b = 0).
- **Recursive case:** the function calls itself with a smaller problem (here, gcd(b, a % b)).

4. Algorithm

Step 1: Start.

Step 2: Read two positive integers, num1 and num2.

Step 3: Call the function gcd(num1, num2).

Step 4: Inside gcd(a, b): if b is 0, return a (base case).

Step 5: Otherwise, return gcd(b, a % b) (recursive case).

Step 6: Display the value returned as the GCD.

Step 7: Stop.

5. Program (C Language)

```c
#include <stdio.h>

/* Recursive function to find GCD using Euclid's algorithm */
int gcd(int a, int b)
{
    if (b == 0)             /* Base case */
        return a;
    return gcd(b, a % b);   /* Recursive case */
}

int main(void)
{
    int num1, num2, result;

    printf("Enter two positive integers: ");
    scanf("%d %d", &num1, &num2);

    if (num1 <= 0 || num2 <= 0)
    {
        printf("Error: Please enter positive integers only.\n");
        return 1;
    }

    result = gcd(num1, num2);
    printf("GCD of %d and %d is %d\n", num1, num2, result);

    return 0;
}
```

6. Explanation of the Code

- **gcd(int a, int b):** the recursive function. When b becomes 0, the answer is a. Otherwise it calls itself with the arguments (b, a % b).
- **main():** reads two integers from the user, checks that both are positive, calls gcd() and prints the result.
- **Input validation:** if either number is zero or negative, an error message is displayed and the program exits.

7. Dry Run (Trace of Recursive Calls)

Example 1: gcd(48, 18)

Call	a	b	Action
1	48	18	b ≠ 0, call gcd(18, 48 % 18 = 12)
2	18	12	b ≠ 0, call gcd(12, 18 % 12 = 6)
3	12	6	b ≠ 0, call gcd(6, 12 % 6 = 0)
4	6	0	b = 0, return 6 (base case)

The value 6 is returned back through every call, so GCD(48, 18) = 6.

Example 2: gcd(270, 192)

Call	a	b	Action
1	270	192	b ≠ 0, call gcd(192, 270 % 192 = 78)
2	192	78	b ≠ 0, call gcd(78, 192 % 78 = 36)
3	78	36	b ≠ 0, call gcd(36, 78 % 36 = 6)
4	36	6	b ≠ 0, call gcd(6, 36 % 6 = 0)
5	6	0	b = 0, return 6 (base case)

Hence, GCD(270, 192) = 6.

8. Sample Output

The program was compiled with gcc and tested with the following inputs:

Test	Input 1	Input 2	Output
1	48	18	GCD of 48 and 18 is 6
2	17	5	GCD of 17 and 5 is 1
3	100	75	GCD of 100 and 75 is 25
4	270	192	GCD of 270 and 192 is 6
5	-4	6	Error: Please enter positive integers only.

Sample run on the console:

```
Enter two positive integers: 48 18
GCD of 48 and 18 is 6
```

9. Complexity Analysis

- **Time complexity:** O(log(min(a, b))), because the remainder shrinks quickly with every call.
- **Space complexity:** O(log(min(a, b))), due to the recursive call stack.

10. Conclusion

The GCD of two positive integers was successfully found using a recursive function based on Euclid's algorithm. The program uses a simple base case (b = 0) and a recursive case (gcd(b, a % b)). It gives correct results for all tested inputs and shows how recursion can solve a problem in a short and elegant way.